side effects of reading from the file, and only if we examine the entire stream and reach its end will we close the file. Although lazy I/O is appealing in that it lets us recover the compositional style to some extent, it's problematic for several reasons:

- It isn't resource-safe. The resource (in this case, a file) will be released only if we traverse to the end of the stream. But we'll frequently want to terminate traversal early (here, `exists` will stop traversing the `Stream` as soon as it finds a match) and we certainly don't want to leak resources every time we do this.
- Nothing stops us from traversing that same `Stream` again, after the file has been closed. This will result in one of two things, depending on whether the `Stream` *memoizes* (caches) its elements once they're forced. If they're memoized, we'll see excessive memory usage since all of the elements will be retained in memory. If they're not memoized, traversing the stream again will cause a read from a closed file handle.
- Since forcing elements of the stream has I/O side effects, two threads traversing a `Stream` at the same time can result in unpredictable behavior.
- In more realistic scenarios, we won't necessarily have full knowledge of what's happening with the `Stream[String]`. It could be passed to some function we don't control, which might store it in a data structure for a long period of time before ever examining it. Proper usage now requires some out-of-band knowledge: we can't just manipulate this `Stream[String]` like a typical pure value—we have to know something about its origin. This is bad for composition, where we shouldn't have to know anything about a value other than its type.

15.2 Simple stream transducers

Our first step toward recovering the high-level style we're accustomed to from `Stream` and `List` while doing I/O is to introduce the notion of *stream transducers* or *stream processors*. A stream transducer specifies a transformation from one stream to another. We're using the term *stream* quite generally here to refer to a sequence, possibly lazily generated or supplied by an external source. This could be a stream of lines from a file, a stream of HTTP requests, a stream of mouse click positions, or anything else. Let's consider a simple data type, `Process`, that lets us express stream transformations.[3]

Listing 15.2 The `Process` data type

```
sealed trait Process[I,O]

case class Emit[I,O](
    head: O,
    tail: Process[I,O] = Halt[I,O]())
  extends Process[I,O]
```

We use a default argument so that we can say `Emit(xs)` as shorthand for `Emit(xs, Halt())`.

[3] We've chosen to omit variance annotations in this chapter for simplicity, but it's possible to write this as `Process[-I,+O]`. We're also omitting some trampolining that would prevent stack overflows in certain circumstances. See the chapter notes for discussion of more robust representations.